

SET 15

SQL> CREATE TABLE EMP

```
2 (  
3   EMPNO NUMBER(5) PRIMARY KEY,  
4   ENAME VARCHAR2(20),  
5   SAL NUMBER(10)  
6 );
```

Table created.

SQL> INSERT INTO EMP VALUES(101,'RAM',20000);

1 row created.

SQL>

SQL> COMMIT;

Commit complete.

SQL> SET SERVEROUTPUT ON;

SQL>

SQL> DECLARE

```
2   v_empno NUMBER(5);  
3   v_ename VARCHAR2(20);  
4   v_sal   NUMBER(10);  
5   v_count NUMBER;  
6 BEGIN  
7   v_empno := &EMPNO;  
8   v_ename := '&ENAME';  
9   v_sal   := &SAL;  
10
```

```

11  SELECT COUNT(*)
12  INTO v_count
13  FROM EMP
14  WHERE EMPNO = v_empno;
15
16  IF v_count = 0 THEN
17
18      INSERT INTO EMP(EMPNO,ENAME,SAL)
19      VALUES(v_empno,v_ename,v_sal);
20
21      DBMS_OUTPUT.PUT_LINE('Record Inserted');
22
23  ELSE
24
25      UPDATE EMP
26      SET ENAME = v_ename,
27          SAL = v_sal
28      WHERE EMPNO = v_empno;
29
30      DBMS_OUTPUT.PUT_LINE('Record Updated');
31
32  END IF;
33
34  COMMIT;
35  END;
36  /

```

Enter value for empno: 102

old 7: v_empno := &EMPNO;

new 7: v_empno := 102;

Enter value for ename: Mansi

old 8: v_ename := '&ENAME';

```
new 8: v_ename := 'Mansi';
```

Enter value for sal: 25000

```
old 9: v_sal := &SAL;
```

```
new 9: v_sal := 25000;
```

Record Inserted

PL/SQL procedure successfully completed.

SET 16

```
SQL> SET SERVEROUTPUT ON;
```

```
SQL> DECLARE
```

```
    num NUMBER;
```

```
    -- User defined exception
```

```
    negative_number EXCEPTION;
```

```
BEGIN
```

```
    -- Accept number from user
```

```
    num := &num;
```

```
    -- Check for negative number
```

```
    IF num < 0 THEN
```

```
        RAISE negative_number;
```

```
    END IF;
```

```
    -- Check odd or even
```

```
    IF MOD(num,2) = 0 THEN
```

```
        DBMS_OUTPUT.PUT_LINE('Number is EVEN');
```

```
    ELSE
```

```
        DBMS_OUTPUT.PUT_LINE('Number is ODD');
```

```
    END IF;
```

EXCEPTION

WHEN negative_number THEN

DBMS_OUTPUT.PUT_LINE('Negative number is not allowed');

END;

/

Enter value for num: 8

old 9: num := #

new 9: num := 8;

Number is EVEN

PL/SQL procedure successfully completed.

SQL> /

Enter value for num: 5

old 9: num := #

new 9: num := 5;

Number is ODD

PL/SQL procedure successfully completed.

SET 17

SQL> CREATE TABLE EMP

2 (

3 EMPNO NUMBER(5),

4 ENAME VARCHAR2(20),

5 SAL NUMBER(10,2),

6 DEPTNO NUMBER(5),

7 COMM NUMBER(10,2)

8);

Table created.

```
SQL> INSERT INTO EMP VALUES(101,'RAM',20000,10,500);
```

1 row created.

```
SQL> INSERT INTO EMP VALUES(102,'SHYAM',25000,20,1000);
```

1 row created.

```
SQL> INSERT INTO EMP VALUES(103,'AMAN',30000,30,1500);
```

1 row created.

```
SQL>
```

```
SQL> COMMIT;
```

Commit complete.

```
SQL> SET SERVEROUTPUT ON;
```

```
SQL> DECLARE
```

```
2   v_empno NUMBER(5);
```

```
3   v_deptno NUMBER(5);
```

```
4   v_sal NUMBER(10,2);
```

```
5   v_comm NUMBER(10,2);
```

```
6
```

```
7 BEGIN
```

```
8   -- Accept EMPNO from user
```

```
9   v_empno := &EMPNO;
```

```
10
```

```
11  -- Get employee details
12  SELECT DEPTNO, SAL, NVL(COMM,0)
13  INTO v_deptno, v_sal, v_comm
14  FROM EMP
15  WHERE EMPNO = v_empno;
16
17  -- Update salary based on DEPTNO
18  IF v_deptno = 10 THEN
19
20      UPDATE EMP
21      SET SAL = SAL + (SAL * 0.10)
22      WHERE EMPNO = v_empno;
23
24      DBMS_OUTPUT.PUT_LINE('Salary updated by 10%');
25
26  ELSIF v_deptno = 20 THEN
27
28      UPDATE EMP
29      SET SAL = SAL + (SAL * 0.15)
30      WHERE EMPNO = v_empno;
31
32      DBMS_OUTPUT.PUT_LINE('Salary updated by 15%');
33
34  ELSE
35
36      UPDATE EMP
37      SET SAL = SAL + NVL(COMM,0)
38      WHERE EMPNO = v_empno;
39
40      DBMS_OUTPUT.PUT_LINE('Salary updated with SAL + COMM');
41
```

```
42  END IF;
43
44  COMMIT;
45
46  EXCEPTION
47  WHEN NO_DATA_FOUND THEN
48      DBMS_OUTPUT.PUT_LINE('Employee not found');
49
50  END;
51  /
```

Enter value for empno: 101

old 9: v_empno := &EMPNO;

new 9: v_empno := 101;

Salary updated by 10%

PL/SQL procedure successfully completed.

SQL> /102

Enter value for empno: 102

old 9: v_empno := &EMPNO;

new 9: v_empno := 102;

Salary updated by 15%

PL/SQL procedure successfully completed.

SQL> /

Enter value for empno: 104

old 9: v_empno := &EMPNO;

new 9: v_empno := 104;

Employee not found

PL/SQL procedure successfully completed.

SET 18

```
SQL> CREATE TABLE STUD
```

```
2 (
```

```
3   ROLL NUMBER(5),
```

```
4   ATT NUMBER(5,2),
```

```
5   STATUS VARCHAR2(5)
```

```
6 );
```

Table created.

```
SQL> INSERT INTO STUD VALUES(1,80,NULL);
```

1 row created.

```
SQL> INSERT INTO STUD VALUES(2,60,NULL);
```

1 row created.

```
SQL> INSERT INTO STUD VALUES(3,75,NULL);
```

1 row created.

```
SQL>
```

```
SQL> COMMIT;
```

Commit complete.

```
SQL> SET SERVEROUTPUT ON;
```

```
SQL>
```



```
SQL> DECLARE

2   v_roll NUMBER(5);

3   v_att NUMBER(5,2);

4

5 BEGIN

6   -- Accept roll number from user

7   v_roll := &ROLL;

8

9   -- Get attendance of student

10  SELECT ATT

11  INTO v_att

12  FROM STUD

13  WHERE ROLL = v_roll;

14

15  -- Check attendance

16  IF v_att < 75 THEN

17

18      UPDATE STUD

19      SET STATUS = 'D'

20      WHERE ROLL = v_roll;

21

22      DBMS_OUTPUT.PUT_LINE('Term not granted');

23

24  ELSE

25

26      UPDATE STUD

27      SET STATUS = 'ND'

28      WHERE ROLL = v_roll;

29

30      DBMS_OUTPUT.PUT_LINE('Term granted');

31
```

```

32  END IF;
33
34  COMMIT;
35
36  EXCEPTION
37  WHEN NO_DATA_FOUND THEN
38      DBMS_OUTPUT.PUT_LINE('Roll number does not exist');
39
40  WHEN OTHERS THEN
41      DBMS_OUTPUT.PUT_LINE('Error occurred');
42
43  END;
44  /

```

Enter value for roll: 2

old 7: v_roll := &ROLL;

new 7: v_roll := 2;

Term not granted

PL/SQL procedure successfully completed.

SQL> /

Enter value for roll: 1

old 7: v_roll := &ROLL;

new 7: v_roll := 1;

Term granted

PL/SQL procedure successfully completed.

SET 19

SQL> CREATE TABLE Borrower

2 (

```
3 Rollin NUMBER(5),
4 Name VARCHAR2(20),
5 DateofIssue DATE,
6 NameofBook VARCHAR2(30),
7 Status CHAR(1)
8 );
```

Table created.

```
SQL> CREATE TABLE Fine
```

```
2 (
3 Roll_no NUMBER(5),
4 Date1 DATE,
5 Amt NUMBER(10)
6 );
```

Table created.

```
SQL> INSERT INTO Borrower
```

```
2 VALUES(1,'SHWETA','01-MAY-2026','DBMS','I');
```

1 row created.

```
SQL>
```

```
SQL> INSERT INTO Borrower
```

```
2 VALUES(2,'MANSI','20-MAY-2026','JAVA','I');
```

1 row created.

```
SQL>
```

```
SQL> COMMIT;
```

Commit complete.

```
SQL> SET SERVEROUTPUT ON;
```

```
SQL>
```

```
SQL> DECLARE
```

```
2   v_roll NUMBER(5);
```

```
3   v_book VARCHAR2(30);
```

```
4   v_date DATE;
```

```
5   v_days NUMBER;
```

```
6   v_fine NUMBER := 0;
```

```
7
```

```
8 BEGIN
```

```
9   -- Accept input from user
```

```
10  v_roll := &ROLL_NO;
```

```
11  v_book := '&BOOK_NAME';
```

```
12
```

```
13  -- Get issue date
```

```
14  SELECT DateofIssue
```

```
15  INTO v_date
```

```
16  FROM Borrower
```

```
17  WHERE Rollin = v_roll
```

```
18  AND NameofBook = v_book;
```

```
19
```

```
20  -- Calculate number of days
```

```
21  v_days := SYSDATE - v_date;
```

```
22
```

```
23  -- Fine calculation
```

```
24  IF v_days BETWEEN 15 AND 30 THEN
```

```
25
```

```
26    v_fine := v_days * 5;
```

```
27
28  ELSIF v_days > 30 THEN
29
30    v_fine := (30 * 5) + ((v_days - 30) * 50);
31
32  END IF;
33
34  -- Insert fine details if fine exists
35  IF v_fine > 0 THEN
36
37    INSERT INTO Fine
38    VALUES(v_roll, SYSDATE, v_fine);
39
40    DBMS_OUTPUT.PUT_LINE('Fine Amount = ' || v_fine);
41
42  ELSE
43
44    DBMS_OUTPUT.PUT_LINE('No Fine');
45
46  END IF;
47
48  -- Update status from I to R
49  UPDATE Borrower
50  SET Status = 'R'
51  WHERE Rollin = v_roll
52  AND NameofBook = v_book;
53
54  COMMIT;
55
56  DBMS_OUTPUT.PUT_LINE('Book Returned');
57
```

```

58 EXCEPTION
59   WHEN NO_DATA_FOUND THEN
60     DBMS_OUTPUT.PUT_LINE('Record not found');
61
62   WHEN OTHERS THEN
63     DBMS_OUTPUT.PUT_LINE('Error occurred');
64
65 END;
66 /

```

Enter value for roll_no: 1

old 10: v_roll := &ROLL_NO;

new 10: v_roll := 1;

Enter value for book_name: DBMS

old 11: v_book := '&BOOK_NAME';

new 11: v_book := 'DBMS';

No Fine

Book Returned

PL/SQL procedure successfully completed.

SQL> /

Enter value for roll_no: 2

old 10: v_roll := &ROLL_NO;

new 10: v_roll := 2;

Enter value for book_name: JAVA

old 11: v_book := '&BOOK_NAME';

new 11: v_book := 'JAVA';

No Fine

Book Returned

PL/SQL procedure successfully completed.

SET 21

SQL> CREATE TABLE EMP

```
2 (  
3   EMPNO NUMBER(5),  
4   ENAME VARCHAR2(20),  
5   SAL NUMBER(10)  
6 );
```

Table created.

SQL> INSERT INTO EMP VALUES(101,'RAM',25000);

1 row created.

SQL> INSERT INTO EMP VALUES(102,'SHYAM',30000);

1 row created.

SQL> INSERT INTO EMP VALUES(103,'AMAN',20000);

1 row created.

SQL> INSERT INTO EMP VALUES(104,'MANSI',40000);

1 row created.

SQL> INSERT INTO EMP VALUES(105,'SHWETA',35000);

1 row created.

SQL>

```
SQL> COMMIT;
```

Commit complete.

```
SQL> SET SERVEROUTPUT ON;
```

```
SQL>
```

```
SQL> DECLARE
```

```
2   v_n NUMBER;
```

```
3
```

```
4   v_empno EMP.EMPNO%TYPE;
```

```
5   v_ename EMP.ENAME%TYPE;
```

```
6   v_sal EMP.SAL%TYPE;
```

```
7
```

```
8   -- Explicit Cursor
```

```
9   CURSOR c1 IS
```

```
10      SELECT EMPNO, ENAME, SAL
```

```
11      FROM EMP
```

```
12      ORDER BY SAL DESC;
```

```
13
```

```
14 BEGIN
```

```
15   -- Accept Top N value
```

```
16   v_n := &N;
```

```
17
```

```
18   OPEN c1;
```

```
19
```

```
20   FOR i IN 1..v_n LOOP
```

```
21
```

```
22      FETCH c1 INTO v_empno, v_ename, v_sal;
```

```
23
```

```
24      EXIT WHEN c1%NOTFOUND;
```

```
25
```



```

26    DBMS_OUTPUT.PUT_LINE
27    (
28        'EMPNO : ' || v_empno ||
29        ' ENAME : ' || v_ename ||
30        ' SALARY : ' || v_sal
31    );
32
33    END LOOP;
34
35    CLOSE c1;
36
37 EXCEPTION
38    WHEN OTHERS THEN
39        DBMS_OUTPUT.PUT_LINE('Error Occurred');
40
41 END;
42 /

```

Enter value for n: 3

old 16: v_n := &N;

new 16: v_n := 3;

EMPNO : 104 ENAME : MANSI SALARY : 40000

EMPNO : 105 ENAME : SHWETA SALARY : 35000

EMPNO : 102 ENAME : SHYAM SALARY : 30000

PL/SQL procedure successfully completed.

SET 22

SQL> CREATE TABLE Stud_Marks

```

2 (
3     Roll NUMBER(5),
4     Name VARCHAR2(20),

```

```
5   Total_Marks NUMBER(5)
6 );
```

Table created.

```
SQL> CREATE TABLE Result
2 (
3   Roll NUMBER(5),
4   Name VARCHAR2(20),
5   Class VARCHAR2(30)
6 );
```

Table created.

```
SQL> INSERT INTO Stud_Marks VALUES(1,'SHWETA',1200);
```

1 row created.

```
SQL> INSERT INTO Stud_Marks VALUES(2,'MANSI',950);
```

1 row created.

```
SQL> INSERT INTO Stud_Marks VALUES(3,'RAM',850);
```

1 row created.

```
SQL>
```

```
SQL> COMMIT;
```

Commit complete.

```
SQL> CREATE OR REPLACE PROCEDURE proc_Grade
```

```
2 (
```

```
3   p_roll NUMBER
```

```
4 )
```

```
5 IS
```

```
6   v_name VARCHAR2(20);
```

```
7   v_marks NUMBER;
```

```
8   v_class VARCHAR2(30);
```

```
9
```

```
10 BEGIN
```

```
11   -- Get student details
```

```
12   SELECT Name, Total_Marks
```

```
13   INTO v_name, v_marks
```

```
14   FROM Stud_Marks
```

```
15   WHERE Roll = p_roll;
```

```
16
```

```
17   -- Categorization
```

```
18   IF v_marks BETWEEN 990 AND 1500 THEN
```

```
19
```

```
20     v_class := 'Distinction';
```

```
21
```

```
22   ELSIF v_marks BETWEEN 900 AND 989 THEN
```

```
23
```

```
24     v_class := 'First Class';
```

```
25
```

```
26   ELSIF v_marks BETWEEN 825 AND 899 THEN
```

```
27
```

```
28     v_class := 'Higher Second Class';
```

```
29
```

```
30   ELSE
```

```
31
```

```

32     v_class := 'Fail';
33
34 END IF;
35
36 -- Insert into Result table
37 INSERT INTO Result
38 VALUES(p_roll, v_name, v_class);
39
40 DBMS_OUTPUT.PUT_LINE('Record Inserted into Result Table');
41
42 EXCEPTION
43 WHEN NO_DATA_FOUND THEN
44     DBMS_OUTPUT.PUT_LINE('Student not found');
45
46 WHEN OTHERS THEN
47     DBMS_OUTPUT.PUT_LINE('Error Occurred');
48
49 END;
50 /

```

Procedure created.

```
SQL> SET SERVEROUTPUT ON;
```

```
SQL>
```

```
SQL> BEGIN
```

```
2   proc_Grade(&ROLL);
```

```
3 END;
```

```
4 /
```

Enter value for roll: 1

```
old 2:  proc_Grade(&ROLL);
```

```
new 2:  proc_Grade(1);
```

Record Inserted into Result Table

PL/SQL procedure successfully completed.